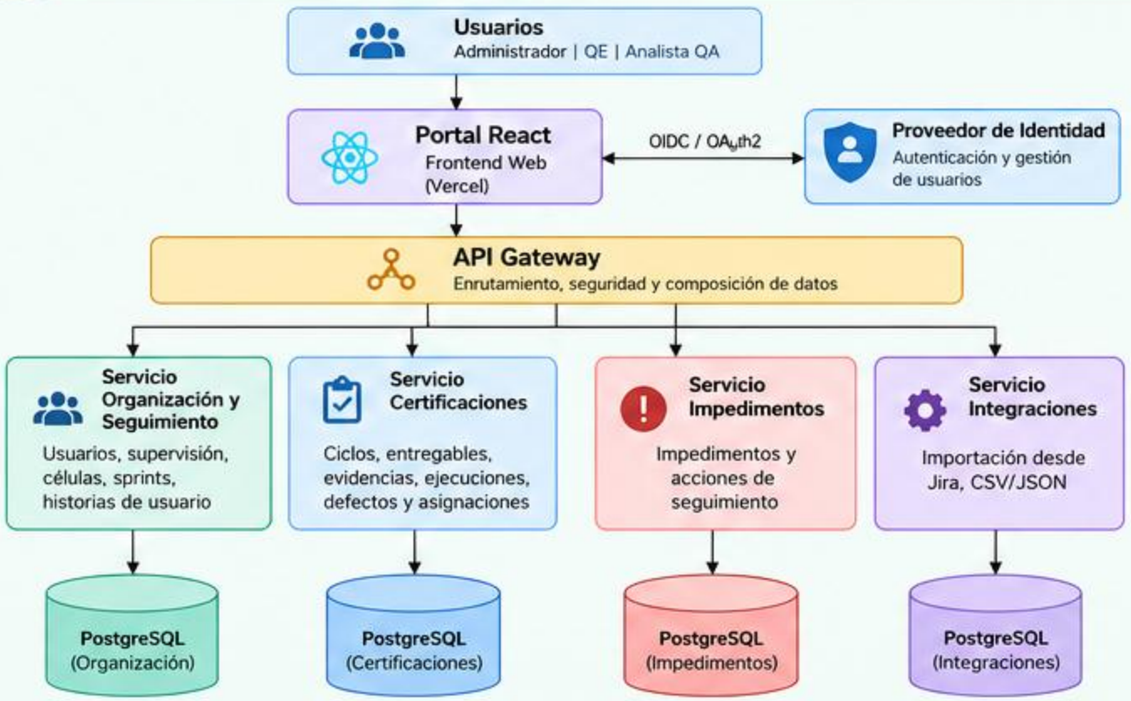
Modelo 4+1 – Quality360

Vista general de la arquitectura de la solución

1 Vista Lógica

¿Qué componentes existen y cómo se relacionan?



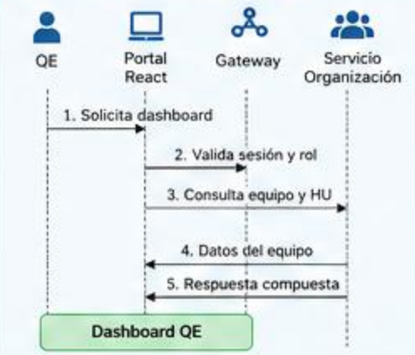
2 Vista de Procesos

¿Cómo interactúan los componentes en tiempo de ejecución?

2.1 Inicio de sesión



2.2 Consulta dashboard QE



2.3 Registro de certificación QA



2.4 Gestión de impedimentos



2.5 Integración de historias(Jira / CSV)



3 Vista de Desarrollo

¿Cómo se organiza el código?

Estructura general del repositorio

```
Quality360/
  apps/
    web/            # Portal React
    gateway/        # API Gateway
  services/
    organizacion/
    certificaciones/
    impedimentos/
    integraciones/
  contracts/        # OpenAPI / mensajes
  Infrastructure/
  tests/
  docs/
```

Estructura de cada servicio

```
services.<nombre>/
  api/              # Controladores y rutas
  aplicacion/       # Casos de uso
  dominio/          # Entidades y reglas de negocio
  infraestructura/  # Repositorios y adaptadores
  migraciones/      # Scripts de base de datos
  pruebas/          # Pruebas unitarias e integración
```

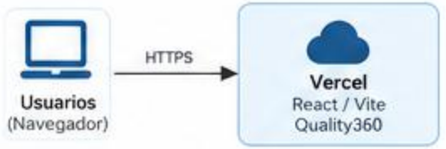
Principios de desarrollo

- Separación por dominios
- El dominio no depende de infraestructura
- Contratos definidos (OpenAPI)
- Pruebas automatizadas
- Despliegue independiente por servicio

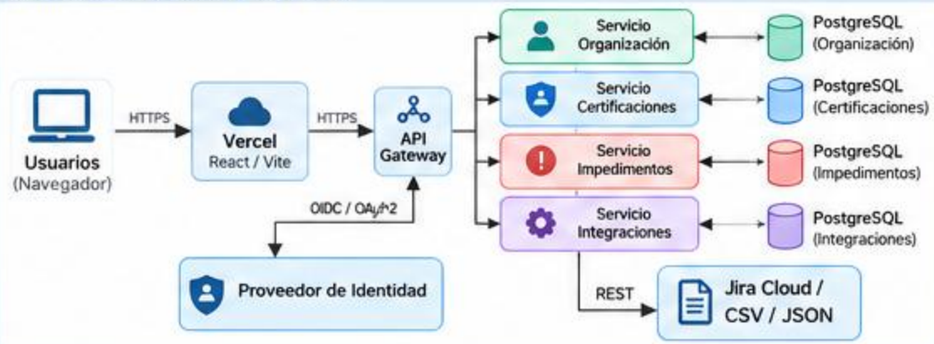
4 Vista Física

¿Dónde se ejecuta la solución?

Situación actual (implementado)



Arquitectura objetivo (propuesta)



5 +1 Vista de Escenarios

¿Qué necesitan hacer los usuarios y sistemas externos?

Actores principales

- **Administrador** (Gestión de usuarios y supervisión)
- **Quality Engineer** (Supervisión y seguimiento del equipo)
- **Analista QA** (Certificación y registro de evidencias)

Sistemas externos

- **Proveedor de identidad** (Autenticación de usuarios)
- **Jira Cloud** (Importación de historias de usuario)
- **CSV / JSON** (Carga de información simulada o anonimizada)

Casos de uso principales

UC01	Iniciar sesión
UC02	Gestionar usuarios (Admin)
UC03	Consultar panel QE
UC04	Consultar historias de usuario
UC05	Registrar certificación QA
UC06	Gestionar entregables y evidencias
UC07	Registrar resultados de pruebas
UC08	Registrar defectos
UC09	Gestionar impedimentos
UC10	Consultar KPI y reportes

Objetivos del modelo

- Separar responsabilidades por dominio
- Garantizar seguridad y control de acceso
- Mantener historial de información
- Facilitar integraciones con Jira y otros sistemas
- Permitir escalabilidad y mantenimiento
- Soportar la evolución futura de la plataforma